

Investigación REACT:

➤ ¿Qué son los estados en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

El estado en React es un objeto o valor interno que contiene información sobre la situación actual de un componente y que puede cambiar con el tiempo y Sirve para crear interfaces dinámicas e interactivas.

Sus características principales son:

- **Son inmutables.**
- **Son locales y privados del componente que lo declara.**
- **Son asíncronicos.**

Ejemplo en código:

```
import React, { useState } from 'react';

function Contador() {
  // 1. Declaración del estado
  const [cuenta, setCuenta] = useState(0);

  // 2. Funciones manejadoras de eventos
  const incrementar = () => {
    setCuenta(cuenta + 1);
  };

  const decrementar = () => {
    setCuenta(cuenta - 1);
  };

  const reiniciar = () => {
    setCuenta(0);
  };

  // 3. Renderizado en pantalla
  return (
    <div style={{ textAlign: 'center', padding: '20px' }}>
      <h2>Contador: {cuenta}</h2>
      <button onClick={decrementar}>-1</button>
      <button onClick={reiniciar} style={{ margin: '0 10px' }}>Reiniciar</button>
      <button onClick={incrementar}>+1</button>
    </div>
  );
}

export default Contador;
```

El hook useState(0) inicializa el estado con el valor 0 y devuelve un arreglo con dos elementos: la variable cuenta, que guarda el valor actual visible en pantalla, y la función setCuenta, encargada de actualizarlo de forma

inmutable. Al hacer clic en cualquiera de los botones, se ejecutan las funciones manejadoras (incrementar, decrementar o reiniciar), las cuales invocan a setCuenta pasándole el nuevo valor; esto le avisa a React que los datos cambiaron, lo que dispara automáticamente un nuevo renderizado del componente Contador y actualiza de inmediato el número mostrado en la interfaz.

➤ ¿Qué son los eventos en React? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo.

Los eventos en React son el mecanismo mediante el cual la aplicación detecta y responde a las interacciones del usuario y sirven para hacer que la interfaz sea interactiva y reactiva.

Sus características principales son:

- **Se pueden pasar funciones como manejadores.**
- **Nomenclatura camelCase.**
- **Hay un único oyente de eventos en la raíz del árbol de renderizado en lugar de asignar oyentes individuales a cada nodo del DOM, optimizando el uso de memoria.**

Ejemplo código:

```

import React, { useState } from 'react';

function SaludoInteractivo() {
  const [nombre, setNombre] = useState('');
  const [mensaje, setMensaje] = useState('');

  // Manejador del evento de escritura en el input
  const manejarCambio = (e) => {
    setNombre(e.target.value);
  };

  // Manejador del evento de clic en el botón
  const manejarClick = () => {
    if (nombre.trim() !== '') {
      setMensaje(`¡Hola, ${nombre}! Bienvenido.`);
    }
  };

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <input
        type="text"
        placeholder="Escribe tu nombre..."
        value={nombre}
        onChange={manejarCambio}
      />
      <button onClick={manejarClick} style={{ marginLeft: '10px' }}>
        Saludar
      </button>

      {mensaje && <h3>{mensaje}</h3>}
    </div>
  );
}

export default SaludoInteractivo;

```

El componente utiliza dos eventos clave para enlazar la acción del usuario con el estado de la aplicación: el evento `onChange` en el `<input>` se dispara cada vez que se presiona una tecla, enviando el objeto de evento sintético `e`, desde el cual `manejarCambio` extrae el texto actual mediante `e.target.value` y actualiza la variable `nombre`. Por su parte, el botón escucha el evento `onClick`, ejecutando `manejarClick` para leer el valor guardado y actualizar el estado `mensaje`, lo que provoca que React vuelva a renderizar la vista y muestre el saludo personalizado en pantalla.

➤ ¿Qué son los Hook en React? ¿Cuáles son sus características y para qué sirve?
¿Cuáles son los principales Hook? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descripto, explicando el ejemplo

Los Hooks son funciones especiales que permiten "enganchar" el estado local y los ciclos de vida de React desde componentes funcionales, sin necesidad de escribir componentes basados en clases, sirven para no tener que reescribir como una clase los componentes funcionales en casos en los que estos necesitaran manejar estado o ejecutar efectos secundarios

Sus características principales son:

- **Solo se utilizan en componentes funcionales.**
- **No se deben llamar dentro de condicionales, bucles o funciones anidadas.**
- **Permiten crear Hooks personalizados agrupando Hooks nativos para encapsular y compartir lógica de negocio específica.**

```

import React, { useState, useEffect } from 'react';

function Contador() {
  // 1. Hook useState: Declara una variable de estado 'cuenta' con valor inicial 0
  // y la función 'setCuenta' para actualizarla.
  const [cuenta, setCuenta] = useState(0);

  // 2. Hook useEffect: Ejecuta un efecto secundario cada vez que 'cuenta' cambia.
  useEffect(() => {
    document.title = `Clics: ${cuenta}`;

    // Opcional: Función de limpieza (cleanup).
    // Se ejecuta antes de volver a correr el efecto o al desmontar el componente.
    return () => {
      document.title = 'React App';
    };
  }, [cuenta]); // Array de dependencias: el efecto solo se ejecuta si 'cuenta' se modifica.

  return (
    <div style={{ textAlign: 'center', marginTop: '40px' }}>
      <h2>Contador de clics</h2>
      <p>Has hecho clic <strong>{cuenta}</strong> veces.</p>

      {/* Modificación del estado usando setCuenta */}
      <button onClick={() => setCuenta(cuenta + 1)} style={{ marginRight: '10px' }}>
        Incrementar
      </button>

      <button onClick={() => setCuenta(cuenta - 1)} style={{ marginRight: '10px' }}>
        Decrementar
      </button>

      <button onClick={() => setCuenta(0)}>
        Reiniciar
      </button>
    </div>
  );
}

export default Contador;

```

En este ejemplo, `useState(0)` inicializa el estado local `cuenta` y provee la función `setCuenta` para modificarlo mediante los botones, lo que dispara un nuevo renderizado de la interfaz con el valor actualizado; paralelamente, `useEffect` intercepta cada cambio en la variable `cuenta` definida en su arreglo de dependencias para ejecutar un efecto secundario que sincroniza el título de la pestaña del navegador (`document.title`), retornando además una función de limpieza opcional que restablece dicho título al valor por defecto cuando el componente se desmonta.